

Contents

Contents	1
Introduction	1
Design Principles	1
Entity Relationship Overview	2
Table Definitions	2
Indexing Strategy	6
Sample Data	7
Data Retention and Purge Considerations	8
MySQL Configuration Notes	8

Database Design Document IPO Allotment Verification System Prepared by: Bhavya Bulani
Version 1.2 · July 2026

Contents

- Introduction
- Design Principles
- Entity Relationship Overview
- Table Definitions
- Indexing Strategy
- Sample Data
- Data Retention and Purge Considerations
- MySQL Configuration Notes

Introduction

This document defines the database schema supporting the IPO Allotment Verification System. Version 1.2 targets **MySQL as the primary database from day one**, replacing the SQLite-first approach used in earlier drafts. MySQL was chosen because the system needs to support concurrent multi-user bulk runs (multiple operators triggering parallel worker pools against the same tables) from the start, which SQLite’s single-writer model does not handle well.

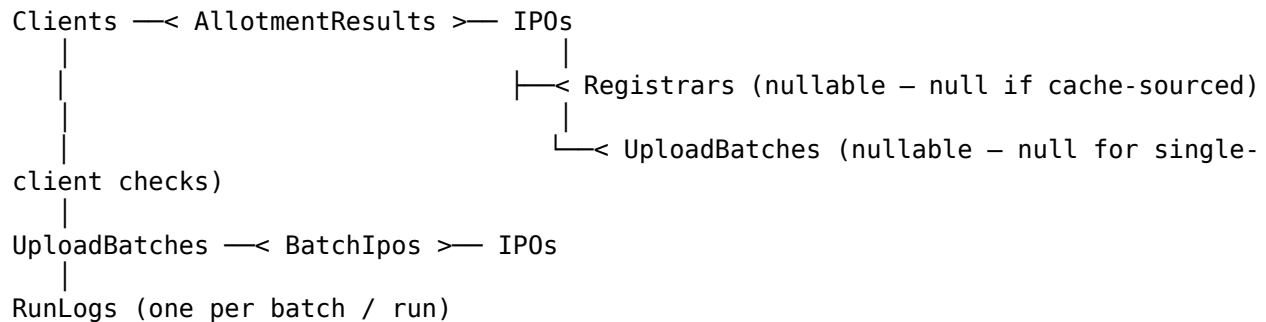
Version 1.1’s three feature additions carry over unchanged in terms of schema *shape*: automatic IPO detection (`auto_detected` / `validated` fields on `ipos`), multi-IPO runs within a single upload batch (the `batch_ipos` join table), and result caching (`cache_expires_at` / `served_from_cache` fields on `allotment_results`). What changes in Version 1.2 is the underlying column types and engine-specific settings, now expressed in MySQL syntax rather than SQLite’s more permissive type affinities.

Design Principles

- Every allotment check result must be traceable back to the client, the IPO, the registrar (or cache) that produced it, and the exact time it was checked.
- Bulk upload batches are tracked as first-class records so a whole batch (and its errors) can be revisited later without re-parsing the original file.
- A single batch may target multiple IPOs at once; the schema must not assume a 1:1 relationship between a batch and an IPO.
- Validation failures are stored, not discarded, so operators can see exactly which rows in their upload failed and why.
- A cached result must be distinguishable from a freshly-checked one, and must carry an explicit expiry so stale data is never served silently past its intended window.

- Auto-detected IPO records must be distinguishable from manually-entered ones and must pass basic validation before being surfaced to users.
- Logs are append-only and separate from operational data, so they can be retained or purged independently.
- All tables use the **InnoDB** storage engine, chosen for foreign-key enforcement, row-level locking (needed for concurrent parallel workers writing results at once), and transactional integrity — MyISAM is not used anywhere in this schema.

Entity Relationship Overview



- One Client can have applications across many IPOs.
- Each AllotmentResult references a client, an IPO, the registrar that supplied (or failed to supply) the result — or null if the result was served from cache — and the upload batch it belongs to (null for single-client checks).
- Each UploadBatch groups many clients together and, via the batch_ipos join table, can target one or more IPOs rather than exactly one.
- Each RunLog captures the operational metadata for one check run (single or bulk, single-IPO or multi-IPO).

Table Definitions

clients

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Internal client record ID
name	VARCHAR(150)	NOT NULL	Client name as provided
pan	CHAR(10)	UNIQUE, NULL	PAN number, 10-character format
client_code	VARCHAR(50)	NULL	Broker-assigned client code
boid_dpid	VARCHAR(50)	NULL	Alternate demat identifier
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Record creation timestamp
updated_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last update timestamp

Engine: InnoDB · Charset: utf8mb4 (to safely store client names with any script/diacritics).

A client is uniquely identifiable by PAN where available; Client Code / BOID-DPID serve as fallback identifiers when PAN is not supplied.

ipos

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Internal IPO record ID
external_id	VARCHAR(100)	UNIQUE, NULL	The IPO's identifier from the external sync source (e.g., Upstox's id field, such as hero-fincorp-ipo). Used to match an incoming synced record to an existing row so re-running the sync updates rather than duplicates it. Null for IPOs entered manually.
name	VARCHAR(200)	NOT NULL	IPO display name (e.g., "NSDL IPO")
open_date	DATE	NULL	IPO subscription open date
close_date	DATE	NULL	IPO subscription close date
status	ENUM('Open','Closed','Not Announced')	NOT NULL	Current IPO status
source	VARCHAR(100)	NULL	Data provider the IPO record was synced from (e.g., Upstox IPO API)
synced_at	DATETIME	NULL	Last sync timestamp
auto_detected	TINYINT(1)	NOT NULL, DEFAULT 0	True if this record was created by the IPO Auto-Detection Service rather than entered manually
validated	TINYINT(1)	NOT NULL, DEFAULT 0	True once the record has passed basic sanity checks and is eligible to appear in the frontend selector

Only rows with validated = 1 are returned by the IPO selection endpoint used to populate the frontend dropdown/multi-select. The IPO Auto-Detection Service matches incoming records against existing rows by external_id, falling back to inserting a new row when no match is

found — this is what keeps repeated sync runs idempotent rather than creating duplicate IPO entries on every run.

status is modeled as a MySQL ENUM rather than a free-text field, since the set of valid values is small and fixed; this also gives a lightweight validation guarantee directly at the database layer.

registrars

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Internal registrar ID
name	VARCHAR(100)	NOT NULL, UNIQUE	Registrar name (Link Intime, KFin Technologies, Bigshare, MUFG Intime)
priority	INT	NOT NULL	Fallback order (1 = tried first)
endpoint_type	ENUM('api','browser_and_notification')	NOT NULL	Integration mechanism
active	TINYINT(1)	NOT NULL, DEFAULT 1	Whether this registrar module is currently enabled

upload_batches

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Internal batch ID
file_name	VARCHAR(255)	NOT NULL	Original uploaded file name
row_count	INT	NOT NULL	Total rows detected in the file
valid_row_count	INT	NOT NULL	Rows that passed validation
invalid_row_count	INT	NOT NULL	Rows that failed validation
uploaded_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Upload timestamp
status	ENUM('Queued','In Progress','Completed','Failed')	NOT NULL	Batch processing status

Note: the single ipo_id foreign key present in Version 1 has been removed from this table. A batch may now target multiple IPOs; the relationship is expressed through the batch_ipos join table below.

batch_ipos

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Internal join-record ID
batch_id	INT	FK → upload_batches.id, NOT NULL, ON DELETE CASCADE	Batch this row belongs to
ipo_id	INT	FK → ipos.id, NOT NULL, ON DELETE RESTRICT	One of the IPOs selected for this batch

A `UNIQUE(batch_id, ipo_id)` constraint prevents the same IPO from being attached to a batch more than once. `ON DELETE CASCADE` on `batch_id` means deleting a batch (per the retention policy) automatically cleans up its IPO associations; `ON DELETE RESTRICT` on `ipo_id` prevents an IPO from being deleted while it is still referenced by an active batch.

allotment_results

Column	Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Internal result ID (BIGINT — this is the highest-volume table)
client_id	INT	FK → clients.id, NOT NULL	Client this result belongs to
ipo_id	INT	FK → ipos.id, NOT NULL	IPO checked
batch_id	INT	FK → upload_batches.id, NULL	Null for single-client checks
registrar_id	INT	FK → registrars.id, NULL	Registrar that produced the result (null if all failed, or served from cache)
status	ENUM('Allotted', 'Not Allotted', 'Website Error', 'Invalid PAN', 'Timeout', 'Server Busy')	NOT NULL	Result category
checked_at	DATETIME	NOT NULL	Timestamp of this specific check
served_from_cache	TINYINT(1)	NOT NULL, DEFAULT 0	True if this result row was returned from the Result Cache Layer rather than a live registrar check

Column	Type	Constraints	Description
cache_expires_at	DATETIME	NULL	For cacheable (successful) results, the timestamp after which this result is no longer eligible to be served from cache and must be re-checked live

An index on (client_id, ipo_id, checked_at) (see Indexing Strategy) is used by the cache lookup to find the most recent cacheable result for a given client/IPO pair efficiently. Error-state results (Website Error, Timeout, Server Busy) always have cache_expires_at = NULL, since error states are never served from cache. id uses BIGINT rather than INT because this table accumulates one row per client × IPO × check and is expected to be the fastest-growing table in the system.

run_logs

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Internal log entry ID
batch_id	INT	FK → upload_batches.id, NULL	Associated batch, if bulk
started_at	DATETIME	NOT NULL	Run start time
completed_at	DATETIME	NULL	Run completion time
success_count	INT	NOT NULL, DEFAULT 0	Successful checks in this run
failure_count	INT	NOT NULL, DEFAULT 0	Failed checks in this run
timeout_count	INT	NOT NULL, DEFAULT 0	Checks that timed out
cache_hit_count	INT	NOT NULL, DEFAULT 0	Checks served from the Result Cache Layer instead of a live registrar request
registrars_used	VARCHAR(255)	NULL	Comma-separated summary of registrars invoked

Indexing Strategy

Table	Index	Reason
clients	UNIQUE(pan)	Fast lookup by PAN; prevents duplicate client records
clients	INDEX(client_code)	Fast lookup when PAN is absent

Table	Index	Reason
ipos	UNIQUE(external_id)	Enables the Auto-Detection Service to match incoming synced records to existing rows and upsert instead of duplicating
ipos	INDEX(validated, status)	Fast retrieval of the current selectable IPO list for the frontend selector
batch_ipos	UNIQUE(batch_id, ipo_id)	Prevents duplicate IPO attachment to the same batch; supports fast “which IPOs is this batch checking” lookups
allotment_results	INDEX(client_id, ipo_id, checked_at)	Fast retrieval of a client’s most recent result for a given IPO — the core cache-lookup query
allotment_results	INDEX(batch_id)	Fast retrieval of all results in a batch for export/history
allotment_results	INDEX(cache_expires_at)	Efficient identification/cleanup of expired cache entries
upload_batches	INDEX(uploaded_at)	Efficient history listing, newest first

Note: MySQL (InnoDB) does not support a filtered/partial unique index the way some other engines do, so the cache-lookup index above is a regular composite index rather than a partial unique index — the application layer (via ORDER BY checked_at DESC LIMIT 1) determines the most recent cacheable row for a client/IPO pair, rather than relying on a database-level uniqueness constraint for “most recent.”

Sample Data

clients

id	name	pan	client_code
1	Rahul Shah	ABCDE1234F	—
2	Amit	PQRSX6789A	—
3	Karan	MNOPQ5678L	—

ipos

id	external_id	name	status	auto_detected	validated
4	NULL	NSDL IPO	Allotment Announced	0	1
5	hdb-financialservices-ipo	HDB Financial IPO	Open	1	1
6	hero-fincorp-ipo	Hero Fincorp IPO	Open	1	1

batch_ipos (batch 501 checking against two IPOs at once)

id	batch_id	ipo_id
1	501	4
2	501	5

allotment_results

id	client_id	ipo_id	registrar_id	status	served_from	cache_expire	checked_at
101	1	4	2 (KFin)	Allotted	0	2026-07-27 15:45:00	2026-07-27 11:45:00
102	2	4	1 (Link Intime)	Not Allotted	0	2026-07-27 15:45:03	2026-07-27 11:45:03
103	3	4	3 (Bigshare)	Allotted	0	2026-07-27 15:45:07	2026-07-27 11:45:07
104	1	4	NULL	Allotted	1	2026-07-27 15:45:00	2026-07-27 13:10:00

Row 104 illustrates a repeat check for client 1 against the same IPO a couple of hours later: rather than re-querying KFin, the system served the still-valid cached result from row 101 and recorded a new result row with `served_from_cache = 1` and no registrar attached.

Data Retention and Purge Considerations

- `run_logs` and `allotment_results` grow with every run and should be periodically archived or purged based on an agreed retention window (e.g., 12 months), rather than growing unbounded. A scheduled event (MySQL EVENT) or an external cron job can run this purge.
- Expired cache entries (`cache_expires_at` in the past) do not need separate deletion — they simply stop being eligible for cache hits — but a periodic cleanup job may still purge very old expired rows as part of the general retention policy.
- `clients` records persist across IPOs and runs, since the same client will typically appear across multiple IPO applications over time.
- `ipos` records marked `auto_detected = 1` but never validated (e.g., malformed sync data) may be purged on a shorter cycle since they were never surfaced to users.
- Large deletes against `allotment_results` should be batched (e.g., `DELETE ... LIMIT 5000` in a loop) rather than issued as a single unbounded statement, to avoid long-held InnoDB locks on a high-write table.

MySQL Configuration Notes

- **Engine:** InnoDB on every table, for foreign-key enforcement and row-level locking under concurrent parallel workers.
- **Character set:** `utf8mb4 / utf8mb4_unicode_ci` throughout, to safely store client names and IPO names in any script without truncation or mojibake.
- **Connection pooling:** the application must use a connection pool (e.g., SQLAlchemy's built-in pool, or a dedicated pooler) sized to comfortably cover the configured worker-pool concurrency (see Technical Architecture Document), since each parallel worker may hold a connection while writing a result.

- **Data-access layer:** all data access goes through a single ORM/data-access layer (e.g., SQLAlchemy models) rather than raw, hand-written SQL scattered through the codebase, so schema changes and any future engine-level tuning stay centralized.
- **Timestamps:** MySQL's DATETIME (not TIMESTAMP) is used throughout to avoid the year-2038 range limitation and timezone-conversion side effects that TIMESTAMP introduces; the application layer is responsible for storing consistent (e.g., UTC) values.
- **Auto-increment gaps:** AUTO_INCREMENT values may show gaps after failed inserts or rollbacks — this is normal InnoDB behavior and must not be relied upon for row counting or sequencing logic anywhere in the application.